

RnD Weekly Progress Report

22b0979

22b1031

22b1071

1 Metrics

1.1 Baseline

Direction	Total Loss	ACK Loss	Data Loss	Avg RTT	Jitter
Server → Client	0.02	0.00	0.02	17.25	12.77
Client → Server	0.00	0.00	0.00	15.75	11.95

Table 1: Metrics for baseline network conditions

1.2 50 ms Delay

Direction	Total Loss	ACK Loss	Data Loss	Avg RTT	Jitter
Server → Client	0.00	0.00	0.00	70.74	14.78
Client → Server	0.00	0.00	0.00	78.81	17.44

Table 2: Metrics under 50 ms induced delay

1.3 100 ms Delay

Direction	Total Loss	ACK Loss	Data Loss	Avg RTT	Jitter
Server → Client	0.00	0.00	0.00	118.07	12.85
Client → Server	2.81	0.00	2.81	121.61	15.71

Table 3: Metrics under 100 ms induced delay

1.4 5% Server Receive Loss

Direction	Total Loss	ACK Loss	Data Loss	Avg RTT	Jitter
Server → Client	4.86	4.86	0.00	7.98	2.08
Client → Server	4.90	0.00	4.88	7.28	1.97

Table 4: Metrics under 5% server receive loss

1.5 20% Server Receive Loss

Direction	Total Loss	ACK Loss	Data Loss	Avg RTT	Jitter
Server → Client	19.49	19.49	0.00	7.95	2.12
Client → Server	20.35	0.00	20.35	7.13	2.12

Table 5: Metrics under 20% server receive loss

1.6 50% Server Receive Loss

Direction	Total Loss	ACK Loss	Data Loss	Avg RTT	Jitter
Server → Client	50.88	50.86	0.04	8.12	2.21
Client → Server	49.40	0.00	49.40	7.41	2.35

Table 6: Metrics under 50% server receive loss

1.7 5% Server Send Loss

Direction	Total Loss	ACK Loss	Data Loss	Avg RTT	Jitter
Server → Client	5.11	0.00	5.11	12.19	6.06
Client → Server	4.93	4.93	0.00	11.54	5.98

Table 7: Metrics under 5% server send loss

1.8 20% Server Send Loss

Direction	Total Loss	ACK Loss	Data Loss	Avg RTT	Jitter
Server → Client	21.18	0.00	21.18	8.05	2.06
Client → Server	20.13	20.07	0.07	7.40	2.04

Table 8: Metrics under 20% server send loss

1.9 50% Server Send Loss

Direction	Total Loss	ACK Loss	Data Loss	Avg RTT	Jitter
Server → Client	49.51	0.00	49.51	9.56	2.90
Client → Server	50.36	50.26	0.18	8.59	2.99

Table 9: Metrics under 50% server send loss

1.10 Symmetric Loss 5%

Direction	Total Loss	ACK Loss	Data Loss	Avg RTT	Jitter
Server → Client	9.22	4.71	4.73	8.41	2.41
Client → Server	13.92	4.45	9.70	7.52	2.38

Table 10: Metrics under 5% symmetric loss(5% each while sending and receiving at server)

1.11 Symmetric Loss 20%

Direction	Total Loss	ACK Loss	Data Loss	Avg RTT	Jitter
Server → Client	35.44	19.52	19.78	9.92	4.52
Client → Server	36.42	20.42	20.11	9.24	4.61

Table 11: Metrics under 20% symmetric loss(20% each while sending and receiving at server)

1.12 50 ms Delay + 20% server send Loss

Direction	Total Loss	ACK Loss	Data Loss	Avg RTT	Jitter
Server → Client	20.76	0.00	20.76	59.82	4.75
Client → Server	19.13	19.12	0.02	59.66	5.19

Table 12: Metrics under combined delay and server send packet loss

1.13 50 ms Delay + 20% server receive Loss

Direction	Total Loss	ACK Loss	Data Loss	Avg RTT	Jitter
Server → Client	20.15	20.15	0.00	62.12	3.59
Client → Server	19.93	0.00	19.93	59.36	3.14

Table 13: Metrics under combined delay and server receive packet loss

2 Client to Server Transmission

2.1 Packet Transmission

For each ping instruction session, the client generates a unique identifier:

- `sessionId`

Each packet sent to the server contains two fields:

- **Session ID**

- **Sequence Number**

The packet payload format is:

Packet = sessionId, sequenceNumber

Each packet transmission records the send timestamp. The metadata for each outstanding packet is stored in:

```
pendingPings[sequenceNumber] = PendingPing(
    sequenceNumber = sequenceNumber,
    sentTime = sentTime,
    location = currentLoc,
    networkType = networkType
)
```

2.2 Acknowledgement Reception

The server sends an acknowledgement containing:

- Sequence number of the current ACK
- A cumulative bitmap representing ACK status of all packets in the session

ACK = sequenceNumber, bitmap

Bitmap Definition:

- Bitmap is a binary string of fixed length $N = \frac{PingSessionDuration}{InterPingDelay}$, for our case with 15 second long sessions and 30 ms interping time $N = 500$
- Index $i - 1$ corresponds to sequence number i since sequence numbers are 1 indexed and the bitstring is 0 indexed
- Value at index $i - 1$:
 - 1 $\rightarrow$ packet with sequence number i was received by the server
 - 0 $\rightarrow$ packet with sequence number i was not received by the server

Example:

Suppose 10 packets have been sent. The server has received sequences $\{1, 2, 3, 5, 7\}$. The bitmap is:

$\underbrace{1}_{seq=1} \underbrace{1}_{seq=2} \underbrace{1}_{seq=3} \underbrace{0}_{seq=4} \underbrace{1}_{seq=5} \underbrace{0}_{seq=6} \underbrace{1}_{seq=7} \underbrace{0}_{seq=8} \underbrace{0}_{seq=9} \underbrace{0}_{seq=10} \underbrace{0 \dots 0}_{seq=11..500}$

Sequences 4, 6, 8, 9, 10 have bit = 0, indicating the packet never reached the server (Data Loss). If the client did not receive an ACK for sequence 3 but its bit = 1, the ACK was lost on the return path (ACK Loss).

Once an acknowledgement is received:

- The corresponding entry in `pendingPings` is removed
- Last Bitmap received is used to infer loss type of all unacknowledged packets

2.3 Round Trip Time (RTT)

RTT measures the time taken for a packet to travel to the server and back.

Let

- T_s = packet send timestamp (`ping.sentTime`)
- T_r = acknowledgement receive timestamp

RTT is computed as:

$$RTT = T_r - T_s$$

```
val rtt = System.currentTimeMillis() - ping.sentTime
```

Average RTT:

$$RTT_{avg} = \frac{\text{totalRtt}}{\text{packetsReceived}}$$

2.4 Packet Loss Classification

At session end, all packets that were never acknowledged are classified using the last received bitmap. Let S be total packets sent, R be total packets acknowledged, and $L = S - R$ be total lost packets. Let $\mathcal{L}$ denote the set of sequence numbers of all lost packets:

$$\mathcal{L} = \{\text{seq} : \text{no ACK received for packet seq}\}$$

2.4.1 Total Loss

$$\text{Total Loss (\%)} = \frac{|\mathcal{L}|}{S} \times 100$$

2.4.2 Data Loss (Forward Path Loss)

A lost packet is classified as Data Loss if its bit in the final bitmap is 0, meaning the packet never reached the server.

$$\text{Data Loss (\%)} = \frac{|\{s \in \mathcal{L} : \text{bitmap}[s - 1] = 0\}|}{S} \times 100$$

2.4.3 ACK Loss (Reverse Path Loss)

A lost packet is classified as ACK Loss if its bit in the final bitmap is 1, meaning the packet reached the server but the ACK was lost on the return path.

$$\text{ACK Loss (\%)} = \frac{|\{s \in \mathcal{L} : b_s = 1\}|}{\sum_{i=1}^S b_i} \times 100$$

The three quantities satisfy:

$$\text{Total Loss} = \text{Data Loss} + \left(\frac{\sum_{i=1}^S b_i}{S} \right) \text{ACK Loss}$$

2.5 Jitter

Jitter represents the variation in RTT:

$$J_i = |RTT_i - RTT_{i-1}|$$

$$J_{avg} = \frac{1}{N-1} \sum_{i=2}^N |RTT_i - RTT_{i-1}|$$

3 Server to Client Transmission

3.1 Session Initialization

Client opens a UDP socket:

```
udpListenerSocket = DatagramSocket(0)
```

OS assigns port:

```
udpListenerPort = udpListenerSocket?.localPort
```

Client sends:

READY Message = $\{device_id, client_id, message, listener_port, session_id\}$

Multiple transmissions improve reliability.

3.2 NAT Keep-Alive Mechanism

Client sends periodic:

KEEP_ALIVE

to maintain NAT mappings.

3.3 Ping Packet Structure

PING Packet = $\{message, sequence\}$

Server stores:

```
self.ping_times[(client_addr, seq)] = (send_time_ms, session_id)
```

3.4 Acknowledgement Packet Structure

Client responds with enhanced ACK:

$$\text{ACK Packet} = \{device_id, client_id, message, sequence, bitmap\}$$

Example:

```
{
  "device_id": "...",
  "client_id": "...",
  "message": "ACK",
  "sequence": 17,
  "bitmap": "11101101000...0"
}
```

All the metrics i.e. RTT , jitter, ACK Loss , Data Loss, Total Loss and bitmap are maintained as earlier.

4 Metrics

4.1 Round Trip Time (RTT)

For each UDP packet:

- The sender records the timestamp when the UDP packet is transmitted, denoted as T_1 .
- The receiver records the timestamp when the packet is received, denoted as T_2 .
- The receiver sends an acknowledgement and records the timestamp when the ACK is transmitted, denoted as T_3 .
- The sender records the timestamp when the ACK is received, denoted as T_4 .

Using these four timestamps, RTT is computed as:

$$RTT = T_4 - T_1$$

Both client and server independently compute RTT:

- Client to Server RTT
- Server to Client RTT

4.2 Packet Loss

Packet loss is determined using a timeout-based detection mechanism combined with bitmap-based loss classification.

For every UDP packet sent:

- The sender waits for an acknowledgement within a fixed timeout interval.
- The timeout interval is set to 10 seconds.
- If the acknowledgement is not received within the timeout, the packet is classified as lost and further categorised using the bitmap into Data Loss or ACK Loss.

Packet loss percentage is calculated as:

$$\text{Total Loss (\%)} = \frac{\text{Number of Lost Packets}}{\text{Total Number of Packets Sent}} \times 100$$

$$\text{Data Loss (\%)} = \frac{\text{Lost packets with bitmap} = 0}{\text{Total Number of Packets Sent}} \times 100$$

$$\text{ACK Loss (\%)} = \frac{\text{Lost packets with bitmap} = 1}{\text{Total Number of ACKs Sent}} \times 100$$

This is measured separately for:

- Client to Server direction
- Server to Client direction

4.3 Jitter

Jitter represents variation in RTT between consecutive packets.

$$Jitter_i = |RTT_i - RTT_{i-1}|$$

$$J_{avg} = \frac{1}{N-1} \sum_{i=2}^N |RTT_i - RTT_{i-1}|$$

Jitter is computed independently for:

- Client to Server direction
- Server to Client direction

5 Summary of Measurements Across Different Network Conditions

Experiment conducted with ping interval increased from 50 ms to 200 ms under different network impairments using `tc`.

5.1 Baseline Measurements (No tc Impairment)

Side	Average RTT	Average Packet Loss
Client to Server	30.97 ms	2.03%
Server to Client	26.24 ms	2.10%

5.2 Artificial Packet Loss (20%)

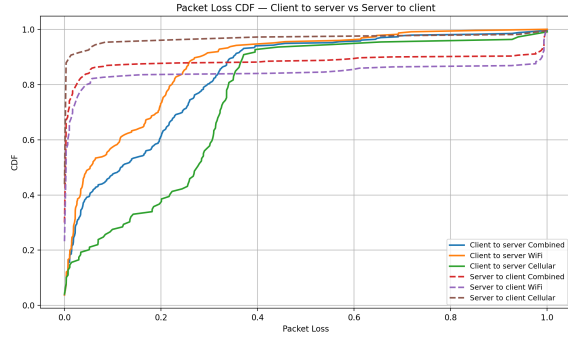
Side	Average RTT	Average Packet Loss
Client to Server	34.40 ms	21.73%
Server to Client	32.64 ms	19.35%

5.3 Artificial Delay (100 ms)

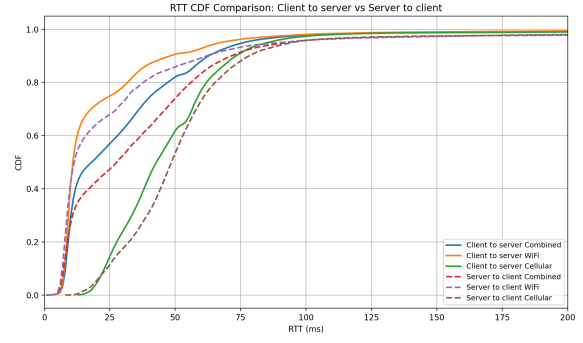
Side	Average RTT	Average Packet Loss
Client to Server	120.58 ms	3.91%
Server to Client	154.99 ms	2.85%

6 CDF Plots

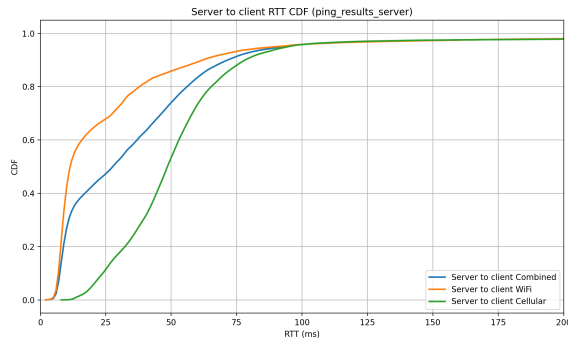
7 Time Series Plots



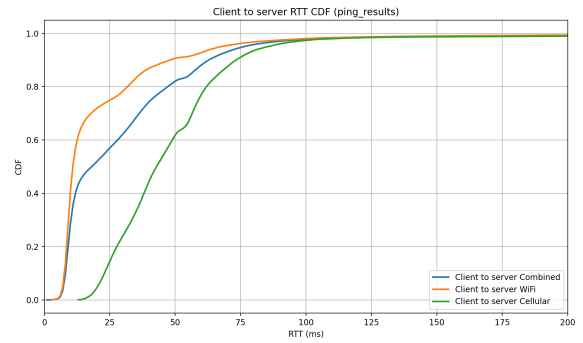
(a) Packet Loss CDF (Client to Server and Server to Client)



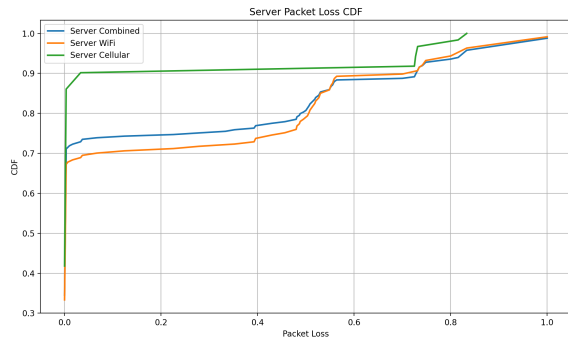
(b) RTT CDF (Client to Server and Server to Client)



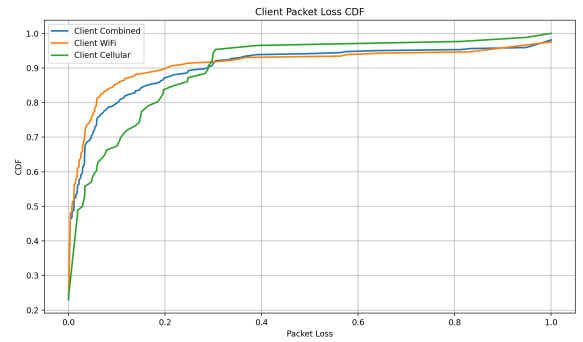
(c) RTT CDF (Server to Client)



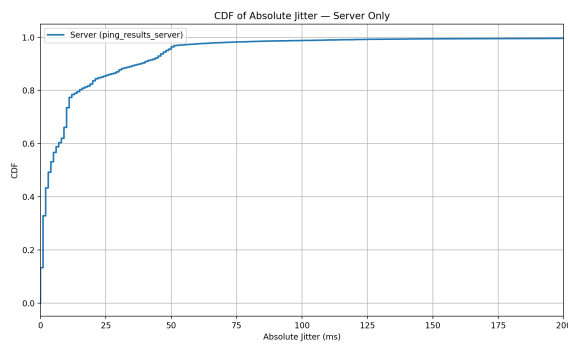
(d) RTT CDF (Client to Server)



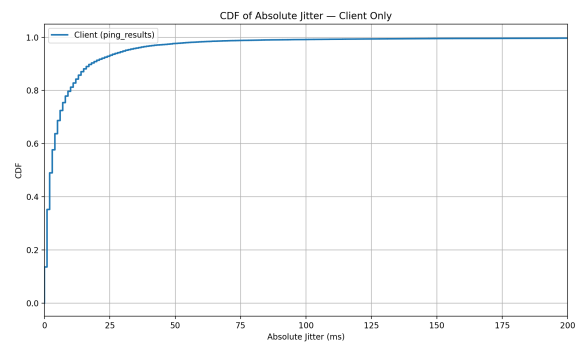
(e) Packet Loss CDF (Server to Client)



(f) Packet Loss CDF (Client to Server)



(g) Jitter CDF (Server to Client)



(h) Jitter CDF (Client to Server)

Figure 1: CDF plots for RTT, Packet Loss, and Jitter

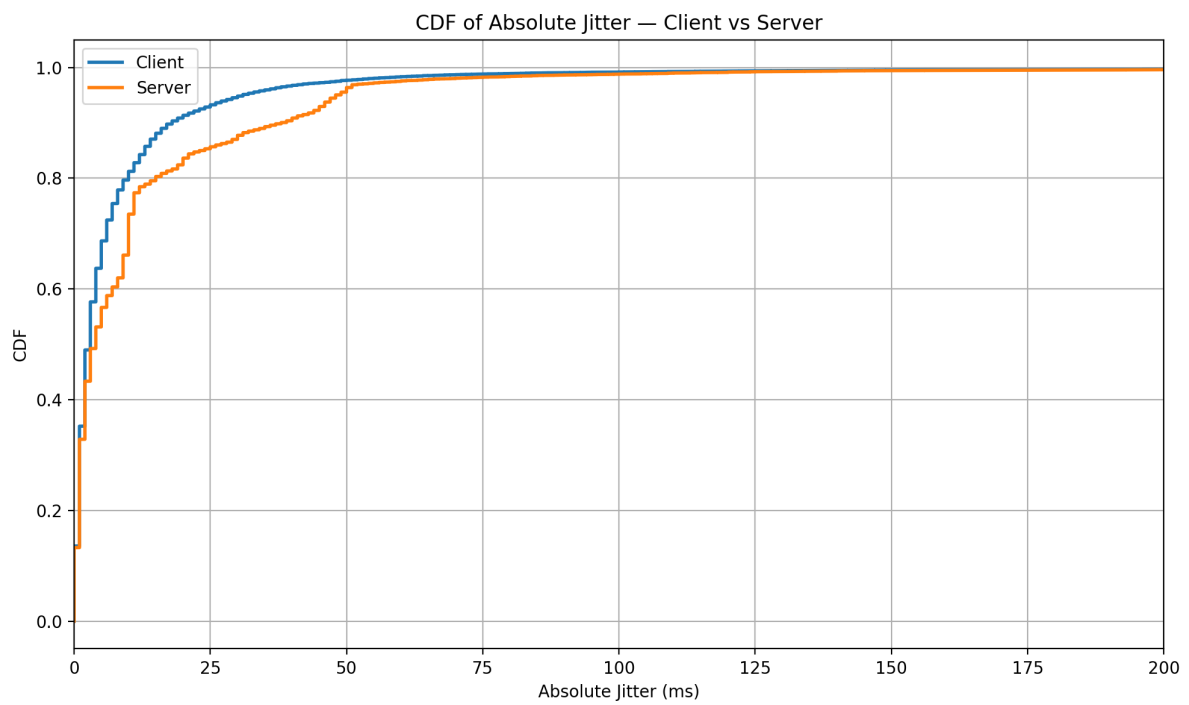


Figure 2: Jitter CDF Comparison (Client to Server and Server to Client)

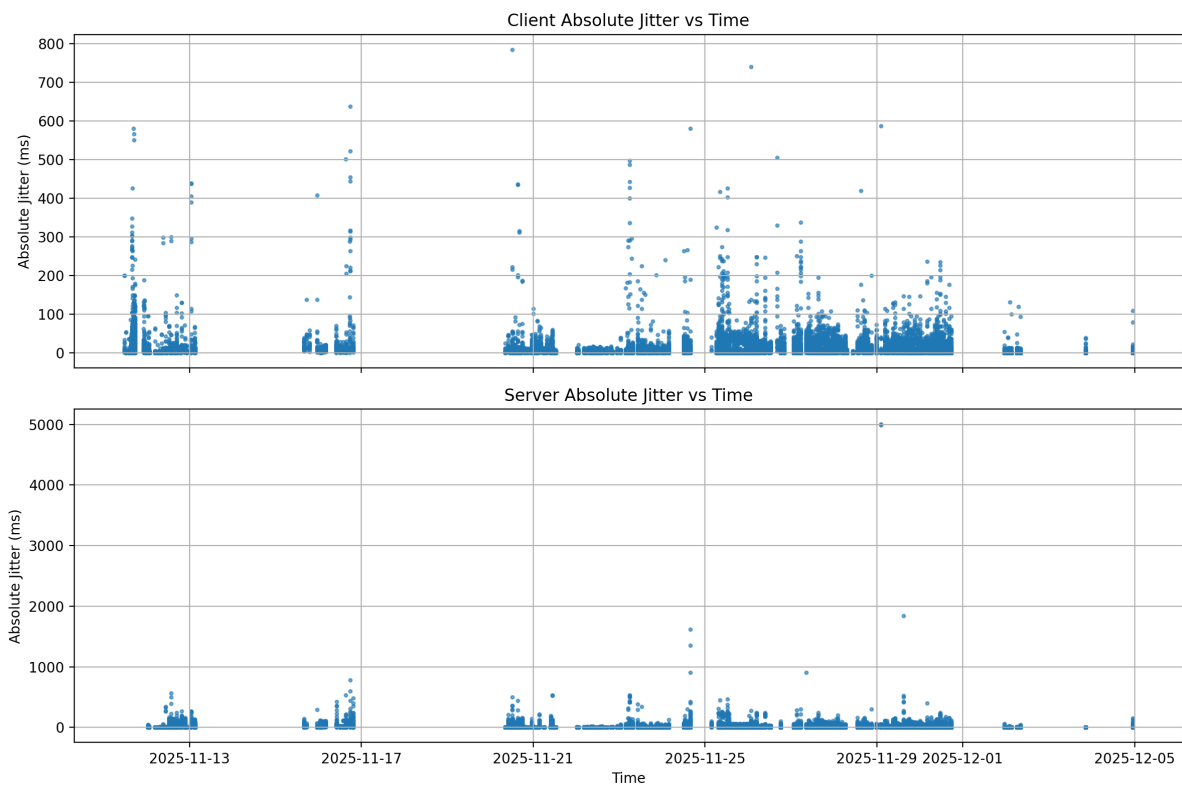


Figure 3: Jitter vs Time (Client to Server and Server to Client)

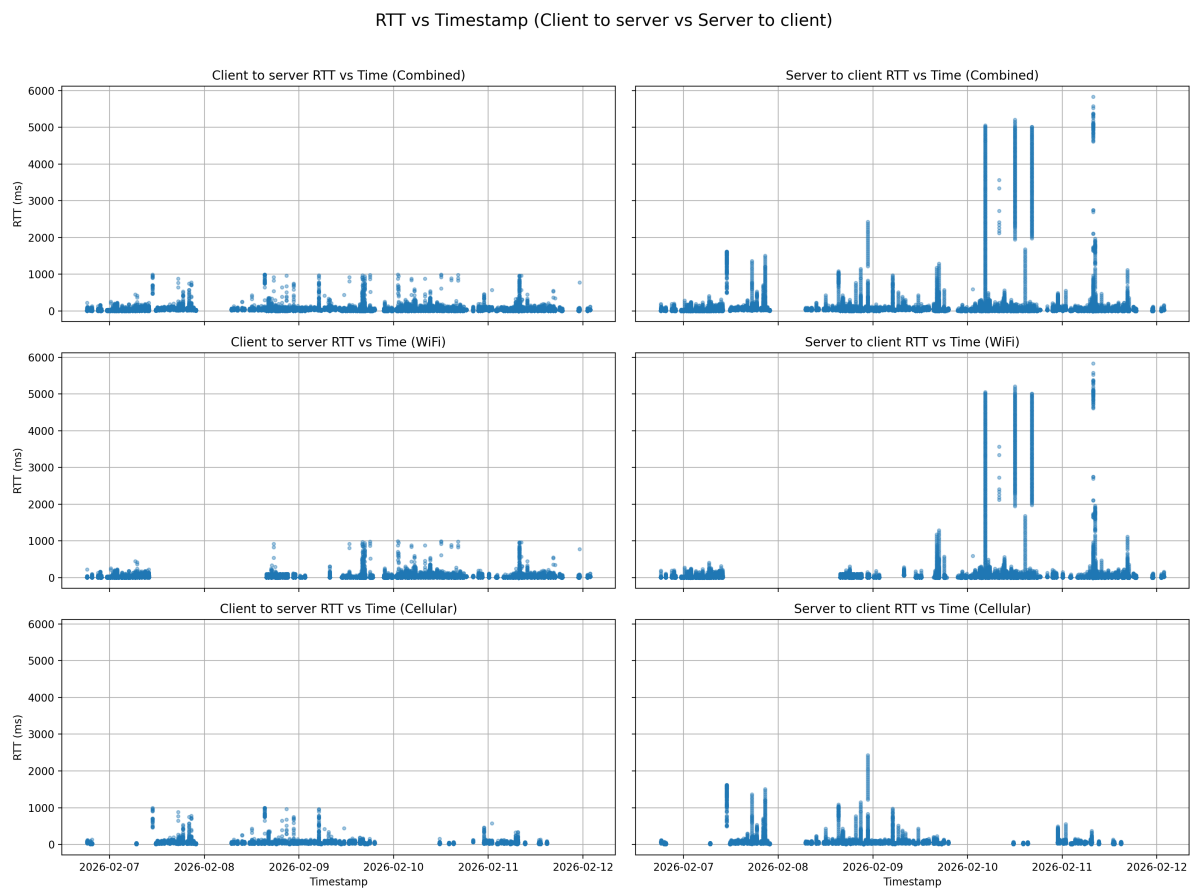


Figure 4: RTT vs Time (Client to Server and Server to Client)

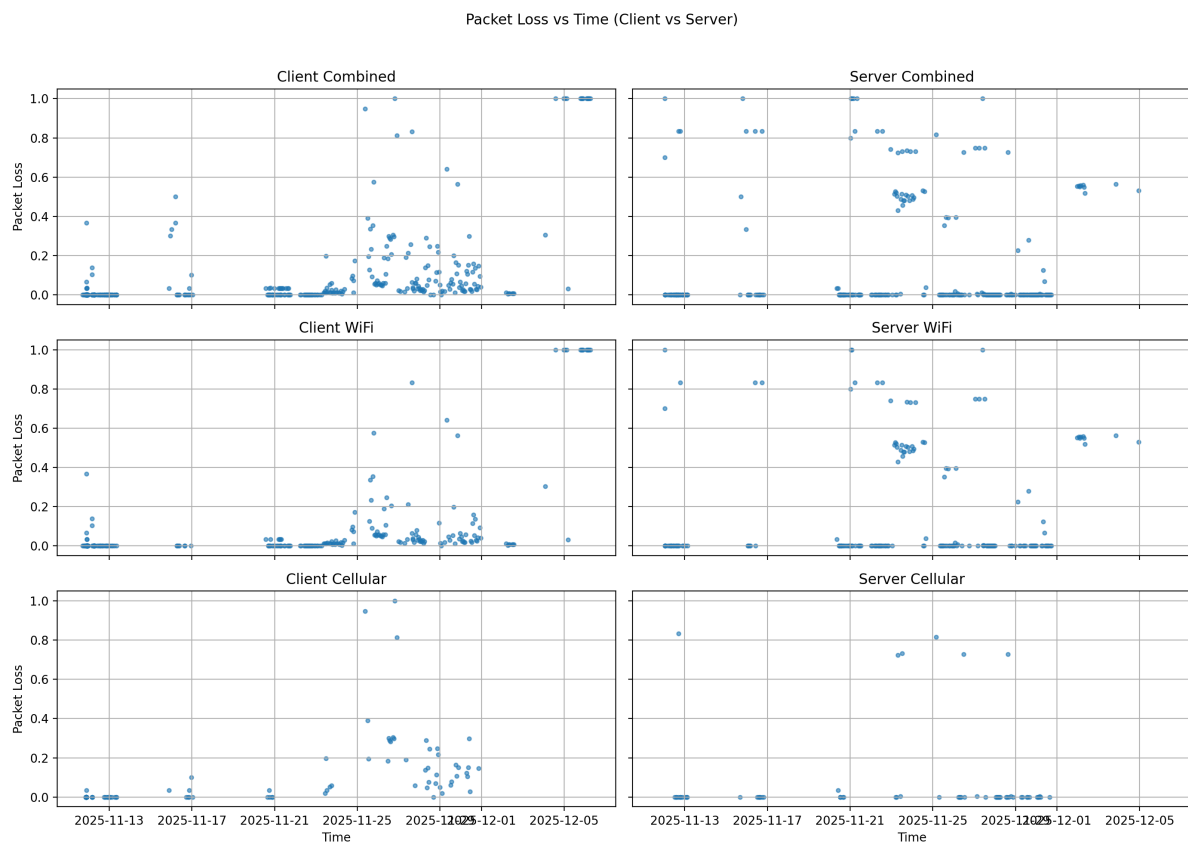


Figure 5: Packet Loss vs Time (Client to Server and Server to Client)